

Aplicación de Gestión de Recursos Humanos

- SPRING BOOT
- SPRING SECURITY
- THYMELEAF
- MYSQL

21/01/2026

**FUTURE
AT HEART**

INDICE

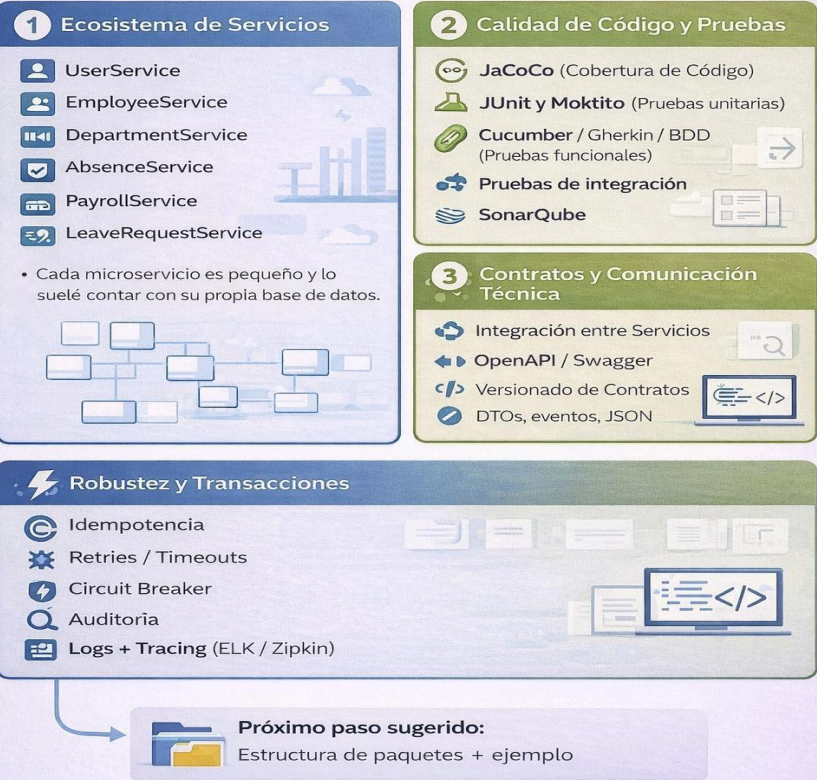
- 00** Descripción del Proyecto
- 01** Ecosistema del proyecto
- 02** Calidad de Código y pruebas
- 03** Comunicación Técnica
- 04** Robustez y Transacciones

00. DESCRIPCIÓN DEL PROYECTO

Aplicación de gestión de Recursos Humanos



Roadmap Técnico: Microservicios RRHH de Alta Calidad



Descripción del proyecto

Desarrolla una aplicación para gestionar empleados, nóminas, permisos, ausencias y tareas dentro de una empresa. Tecnologías: Spring Boot, Spring Security, Thymeleaf, MySQL. Características adicionales:

- Registro de empleados y gestión de roles.
- Gestión de departamentos
- Gestión de permisos y días ausentes.
- Gestión de los pagos y nómina de los empleados.
- Gestionar las solicitudes de permisos o vacaciones de los empleados.

Ecosistema de Servicios

Crea una arquitectura donde cada servicio tenga una responsabilidad única ,garantizando bajo acoplamiento y alta cohesión : UserService, EmployeeService, DepartmentService, AbsenceService, PayrollService, LeaveRequestService

Calidad de Código y pruebas

Crea y aplica estrategias de verificación para asegurar un software profesional : Jacoco, Junit, Mockito, SonarQube, Cucumber

Comunicación técnica

Crea estándares de intercambio robustos : DTOS(Data Transfer Objects) , OpenAPI

Robustez y Transacciones

Crea mecanismos de seguridad y estabilidad para el sistema:

Spring @Transactional : Configura la atomicidad de las operaciones para evitar pérdida de integridad en la base de datos

Manejo Global de Excepciones: Crea un sistema centralizado con @ControllerAdvice para estandarizar las respuestas de error

Logs : Crea un registro detallado de eventos para asegurar la trazabilidad de operaciones críticas

01. Ecosistema del proyecto

Aplicación de Gestión de Recursos Humanos

UserService	DepartmentService
Responsabilidad : Gestión de usuarios dentro del sistema (registro, autenticación, roles, etc.). Funciones: • Registrar un nuevo usuario (por ejemplo, un nuevo administrador o empleado). • Autenticar un usuario mediante nombre de usuario y contraseña. • Cambiar la contraseña del usuario. • Asignar roles a los usuarios (Admin, HR, Empleado, etc.). • Eliminar o desactivar usuarios. • Actualizar usuario • Obtener información del usuario.	Responsabilidad : Gestión de departamentos en la organización. Funciones: • Crear, leer, actualizar y eliminar departamentos. • Asignar un jefe de departamento (responsable). • Ver el listado de empleados por departamento. • Conflictos entre ramas • Modificar la estructura de un departamento (añadir, eliminar empleados). • Ya hecho en el servicio de EmployeeService • Definir los objetivos o proyectos asignados a un departamento. • Manejo de excepciones.
EmployeeService	AbsenceService
Responsabilidad : Gestión de la información relacionada con los empleados. Funciones: • Crear, leer, actualizar y eliminar registros de empleados. • Obtener el perfil de un empleado (información personal, puesto, salario, etc.) • Actualizar la información personal de un empleado. • Asignar o modificar el departamento y puesto de un empleado. • Calcular el salario de los empleados (incluyendo beneficios y deducciones). • Cambiar el estado de un empleado (activo, suspendido, etc.). • Definición nueva entidad para responsable (solución de la relación circular).	Responsabilidad : Gestión de ausencias y permisos de los empleados. Funciones: • Registrar una nueva ausencia (vacaciones, enfermedad, permiso personal, etc.). • Aprobar o rechazar solicitudes de ausencia. • Ver el historial de ausencias de un empleado • Calcular días de ausencia y actualizarlos en el perfil del empleado. • Enviar notificaciones o alertas sobre ausencias (cuando son aprobadas o cuando un empleado supera el límite de ausencias).
PayrollService	LeaveRequestService
Responsabilidad : Gestión de los pagos y nómina de los empleados Funciones : • Calcular y procesar los pagos de los empleados según su salario base, bonificaciones, deducciones, impuestos, etc. • Generar informes de nómina mensuales. • Calcular y mostrar los beneficios y deducciones para cada empleado. • Enviar recibos de pago a los empleados. • Generar informes de costos de personal para la dirección de la empresa. • Gestionar los pagos en diferentes ciclos (semanal, mensual, etc.) .	Responsabilidad : Gestionar las solicitudes de permisos o vacaciones de los empleados. Funciones : • Crear una solicitud de permiso o vacaciones para un empleado. • Aprobar o rechazar solicitudes de permisos. • Ver historial de solicitudes de permisos/vacaciones de un empleado • Notificar al empleado sobre el estado de su solicitud. • Calcular el saldo de días de vacaciones disponibles para un empleado..

02. Calidad de código y pruebas

Aplicación de gestión de Recursos Humanos

Calidad del código



Calidad de Código y Pruebas: Un Enfoque Profesional

1. Fundamentos: Pruebas Unitarias (JUnit & Mockito)

Qué es: Verificación aislada.

Qué : Probar que la lógica funciona y se detecta errores.

Ejemplo: Probar que la función `calculateBaseSalary()`...

Cobertura de Código (JaCoCo)

Qué es: Mide qué porcentaje de código se ejecuta.

Por qué: Identifica áreas sin tests.

Objetivo: Establecer un mínimo (ej. 80%).

Comportamiento: BDD) con Gherkin/Cucumber

Qué es: Describe qué comportamiento debe cumplir el sistema.

Por qué: Asegura que los requisitos se entienden y se validan.

```

"""gherkin
Feature: Cálculo de Nómina
Como administrador de RRHH
Quiero calcular la nómina...

Dado que el empleado 'Juan Perez' tiene un
salario base de 2000 EUR
Y el empleado 'Juan Peres' ha tenido 2
días de ausencia...
Cuando se procese la nómina
Entonces el salario bruto debe ser 1800 EUR
Y se debe generar un recibo...
)

```

Auditoría Continua (SonarQube)

Qué es: Análisis estático automatizado.

Integración: Se ejecuta de forma continua (CI/CD)

Responsabilidad : Crea y aplica estrategias para asegurar un software profesional:

Jacoco: configura informes de cobertura para validar que la lógica crítica esté protegida.

JUnit y Mockito: crea pruebas unitarias que aíslen el funcionamiento de cada componente

Cucumber/Gherkin : Escribe escenarios de prueba en lenguaje natural que definen el comportamiento del negocio

SonarQube: implementa análisis estático para detectar deudas técnicas y vulnerabilidades.

03. Comunicación técnica

Aplicación de Gestión de Recursos Humanos

Comunicación técnica

Comunicación Técnica: Intercambio de Información Robusto

1. DTOs

Qué es: Verificación que traslada los al.

- Qué es: Desacopla de lo que esté más adelante en el proceso.
- **Ejemplo:** EmployeeDTO con id, nombre y salario (ej. 85%).

2. JaCoCo

Qué es: Mide ejemplos que aceptará.

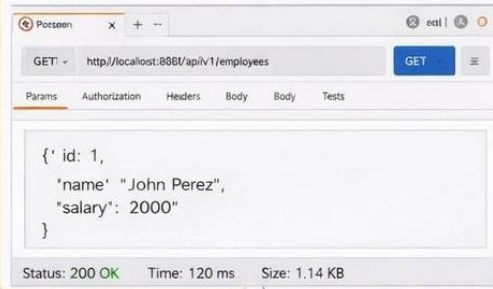
Qué mide: Mide qué se aprovechará el API al codificar filtros y validaciones a partir de lotes totales (ej. 80%).

Objetivo: Establecer un mínimo (ej. 80%).

3. OpenAPI / Swagger

Qué es: Describe lo que lo aceptarán...

Por qué: Es un mecanismo estándar de API REST que proporciona documentación y pruebas.



4. Versionado de Datos { } (JSON / XML)

Qué es: Análisis estático APIs REST.

Por qué: Debido a la evolución de cambios de usuarios puedes utilizar versiones e-datos API REST.

Responsabilidad : Crea estándares de intercambio de información robustos .

DTOS(Data Transfer Objects) : Crea objetos específicos para transferir datos sin exponer directamente las entidades de la base de datos.

OpenAPI/Swagger/PostMan/Pando : Crea la documentación interactiva de todos los puntos de acceso(endpoints) de la API y pruebalos en Postman o Pando

04. Robustez y Transacciones

Aplicación de Gestión de Recursos Humanos

Robustes y Transacciones

Robustez y Transacciones: Asegurando la Estabilidad

1. Transacciones (@Transaccional)

Qué es: Anotación para gestionar transacciones.

Por qué: Asegura que los cambios sean consistentes.

Ejemplo: Métodos anotados con @Transaccional.

2. Manejo Global de Excepciones

Qué es: Control centralizado de errores.

Por qué: Evita fallos sin gestionar y responde a errores comunes.

Ejemplo: Capturar y devolver un mensaje claro ante un fallo.

3. Resiliencia (Circuit Breaker, Retries, Timeouts, Idempotencia)

Qué es: Técnicas para manejar fallos de servicios.

Por qué: Evita caídas por fallos temporales.

Ejemplo: Limitar retiros si una API externa no responde.

4. Logs y Monitorización

Qué es: Registro y análisis de eventos.

Por qué: Detecta problemas y monitorea el rendimiento.

Ejemplo: Logs en Kibana, alertas en Grafana.

Responsabilidad : Crea mecanismos de seguridad y estabilidad para el sistema.

Spring @Transactional : configura la atomicidad de las operaciones para evitar pérdida de integridad en la base de datos.

Manejo Global de Excepciones: crea un sistema centralizado con @ControllerAdvice para estandarizar las respuestas en caso de error.

Logs : Crea un registro detallado de eventos para asegurar la trazabilidad de operaciones críticas. En nuestro caso no vamos a exportarlo ni a Kibana ni a Grafana. Si sería bueno persistirlo en un archivo para verificarlo.

GRACIAS

**FUTURE
AT HEART**